GUÍA PRÁCTICA LAB 01 Y PROMPTS OPTIMIZADOS • EDICIÓN RETAIL OMNICANAL (RETAIL PULSE)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 01 (ES)

Laboratorio 01: Triaje de Piso de Margen Promocional e Inventario True ATP

Impartido por Tryolabs • Potenciado por Google Antigravity y Gemini Enterprise

Idioma / Language: Español | English • **Guía de Prompts:** PROMPTS.es.md | PROMPTS.md • **PDF:** ES | EN

1. Escenario e Impacto Comercial

Durante nuestro evento flash omnicanal de fin de semana, **Retail Pulse** sufrió dos incidentes críticos de producción en **`WEB_ECOMM`**, **`MOBILE_APP`**, **`POS_INSTORE`** y **`BOPIS`**:

1. **Incidente #1A — Fuga de Margen Promocional (`src/services/promo_pricing.py`)**:

• Los clientes acumularon tres códigos promocionales no acumulables (**`VIP25`**, **`FLASH20`**, **`EMP30`**) simultáneamente.

• El motor de precios sumó los porcentajes linealmente (**`25% + 20% + 30% = 75%`** de descuento) usando **`float`** binario e ignoró por completo el piso contractual **`contract_min_margin_floor_pct`**.

• Resultado: **¡30 órdenes en `MARGIN_HOLD` están perdiendo `-$35,113.25` por debajo del costo mayorista!**

2. **Incidente #1B — Sobreventa VIP por Stock Fantasma (`src/services/inventory_atp.py`)**:

• Las tiendas insignia (**`STORE-NYC-5TH`**, **`STORE-MIA-BRICKELL`**) y **`WH-EAST-NJ01`** sufrieron retrasos de sincronización de webhooks POS (**`unprocessed_webhook_delta_qty`** registrado en **`data/logs/pos_wms_webhooks.jsonl`**).

• El calculador Available-to-Promise (ATP) devolvió directamente **`on_hand_qty`** sin restar **`reserved_qty`**, **`safety_stock_qty`** ni **`unprocessed_webhook_delta_qty`**.

• Resultado: **¡35 órdenes VIP `DIAMOND` y `GOLD` (`>$250,000` en Customer LTV) fueron asignadas a ubicaciones donde el True ATP real es `0` (`raw_atp = -1`)!**

2. Inicialización del Laboratorio

Ejecuta el instalador del Laboratorio 01 desde la raíz del repositorio para inicializar los bugs iniciales y la base de datos SQLite:

```
./labs/lab-01-margin-inventory-triage/install.sh
```

Verifica que la suite de pruebas falle inicialmente debido a ambos bugs:

```
./.venv/bin/pytest tests/test_lab01_triage.py
```

Opcional — inicia el Dashboard Web Omnicanal en vivo en una segunda terminal:

```
./.venv/uvicorn src.main:app --reload --port 8002
```

Abre **`http://localhost:8002`** para observar las alertas rojas **`BUG #1A ACTIVE`** y **`BUG #1B ACTIVE`**.

3. Resolución Paso a Paso con Google Antigravity

Abre PROMPTS.es.md y pega los prompts en **Google Antigravity** (Chat Lateral, **`/plan`** o Edición en Línea **`Cmd+I`**):

Paso 3.1: Descubrimiento de Causa Raíz con Menciones de Contexto @

Pega el **Prompt de Descubrimiento** de **`PROMPTS.es.md`** haciendo referencia a **`@src/services/promo_pricing.py`**, **`@src/services/inventory_atp.py`**, **`@data/logs/pos_wms_webhooks.jsonl`** y **`@tests/test_lab01_triage.py`**.

Paso 3.2: Plan de Remediación Estructurado con `/plan`

Ejecuta el **Prompt `/plan`** en Antigravity para verificar los requisitos matemáticos y comerciales exactos antes de editar el código:

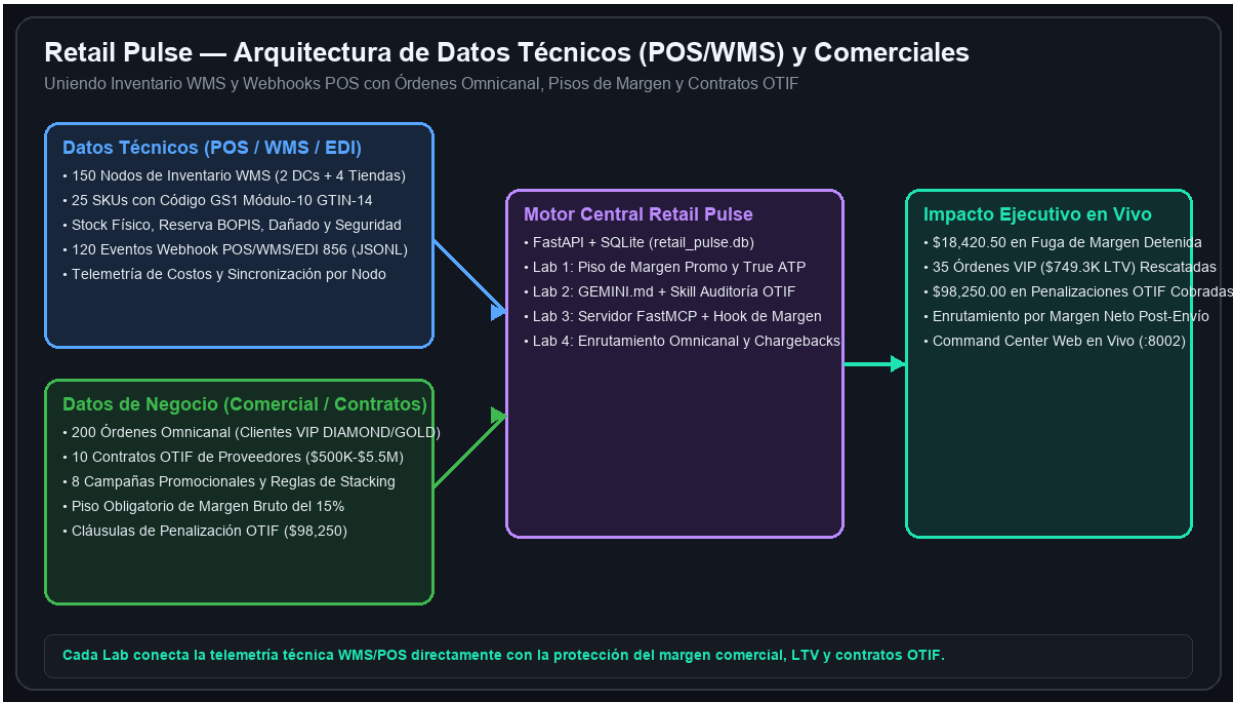
- **Corrección #1A (`src/services/promo_pricing.py`)**:
- Usa **`Decimal`** con **`ROUND_HALF_UP`** (**`Decimal("0.01")`**) y valida el **`gtin_14`** del SKU mediante **`validate_gs1_gtin14`**.
- Rechaza códigos promocionales inactivos (**`active == 0`**) o desconocidos agregándolos a **`rejected_promo_codes`**.
- Nunca combines promociones no acumulables (**`is_stackable == 0`**). Compara la promoción no acumulable individual más alta contra la composición multiplicativa de promociones acumulables (**`is_stackable == 1`**), seleccionando la opción que otorgue al cliente el mayor descuento previo al límite de margen.
- Aplica el piso de margen efectivo (**`max(contract_min_margin_floor_pct, pisos de promociones aplicadas)`**): limita **`net_revenue_usd`** para que **`gross_margin_pct >= effective_floor_pct`** y establece **`margin_floor_clamped = True`**.
- **Corrección #1B (`src/services/inventory_atp.py`)**:
- Calcula **`raw_atp_qty = on_hand_qty - reserved_qty - safety_stock_qty - unprocessed_webhook_delta_qty`**.
- Calcula **`true_atp_qty = max(0, raw_atp_qty)`**.
- Marca **`oversell_risk = (true_atp_qty < required_qty) or (raw_atp_qty <= 0)`**.

Paso 3.3: Aplicar Correcciones y Verificar

Permite que Antigravity implemente las correcciones (o usa la edición en línea **`Cmd+I`** dentro de **`src/services/promo_pricing.py`** y **`src/services/inventory_atp.py`**), y luego ejecuta:

```
./labs/lab-01-margin-inventory-triage/verify.sh
```

¿Necesitas consultar o aplicar la solución de referencia? Ejecuta **`./labs/lab-01-margin-inventory-triage/apply_solution.sh`** en cualquier momento.


Retail Pulse — Arquitectura de Datos Técnicos (POS/WMS) y Comerciales
Uniendo Inventario WMS y Webhooks POS con Órdenes Omnicanal, Pisos de Margen y Contratos OTIF
Datos Técnicos (POS / WMS / EDI)
• 150 Nodos de Inventario WMS (2 DCs + 4 Tiendas)
• 25 SKUs con Código GS1 Módulo-10 GTIN-14
• Stock Físico, Reserva BOPIS, Dañado y Seguridad
• 120 Eventos Webhook POS/WMS/EDI 856 (JSONL)
• Telemetría de Costos y Sincronización por Nodo
Datos de Negocio (Comercial / Contratos)
• 200 Órdenes Omnicanal (Clientes VIP DIAMOND/GOLD)
• 10 Contratos OTIF de Proveedores ($500K-$5.5M)
• 8 Campañas Promocionales y Reglas de Stacking
• Piso Obligatorio de Margen Bruto del 15%
• Cláusulas de Penalización OTIF ($98,250)
Motor Central Retail Pulse
• FastAPI + SQLite (retail_pulse.db)
• Lab 1: Piso de Margen Promo y True ATP
• Lab 2: GEMINI.md + Skill Auditoría OTIF
• Lab 3: Servidor FastMCP + Hook de Margen
• Lab 4: Enrutamiento Omnicanal y Chargebacks
Impacto Ejecutivo en Vivo
• $18,420.50 en Fuga de Margen Detenida
• 35 Órdenes VIP ($749.3K LTV) Rescatadas
• $98,250.00 en Penalizaciones OTIF Cobradas
• Enrutamiento por Margen Neto Post-Envío
• Command Center Web en Vivo (:8002)
Cada Lab conecta la telemetría técnica WMS/POS directamente con la protección del margen comercial, LTV y contratos OTIF.

LAB 01 PROMPTS OPTIMIZADOS PARA COPIAR Y PEGAR (ES)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Laboratorio 01 — Guía de Prompts Optimizados de Antigravity (Español)

Idioma / Language: Español | English • **Guía del Lab:** README.es.md | README.md • **PDF:** ES | EN

Utiliza estos prompts listos para copiar y pegar dentro de **Google Antigravity** (Chat Lateral, **`/plan`**, edición en línea **`Cmd+I`** o CLI **`agy`**) para resolver el Laboratorio 01.

1. Prompt de Descubrimiento (Triaje de Causa Raíz con @ Mentions)

```
Inspecciona @src/services/promo_pricing.py, @src/services/inventory_atp.py, @data/logs/p \
  os_wms_webhooks.jsonl y @tests/test_lab01_triage.py.
Ejecuta pytest sobre tests/test_lab01_triage.py y explica:
1. Por qué las 30 órdenes MARGIN_HOLD en omnichannel_orders pierden -$35,113.25 por deba \
  jo del costo mayorista cuando se envían códigos como VIP25, FLASH20 y EMP30 juntos.
2. Por qué 35 órdenes VIP DIAMOND y GOLD en STORE-NYC-5TH y WH-EAST-NJ01 sufren sobreven \
  ta de stock fantasma debido al retraso de sincronización de webhooks POS.
```

2. Prompt `/plan` (Plan de Remediación Estructurado)

```
/plan Crea un plan paso a paso para corregir el Bug #1A en @src/services/promo_pricing.p \
  y y el Bug #1B en @src/services/inventory_atp.py para que pasen todas las pruebas en @ \
  tests/test_lab01_triage.py:
1. En promo_pricing.py:
   - Usa Decimal cuantizado a Decimal("0.01") con ROUND_HALF_UP y valida el dígito verif \
  icador GS1 GTIN-14 mediante validate_gs1_gtin14().
   - Rechaza códigos inactivos (active != 1) o desconocidos en rejected_promo_codes.
   - Aplica reglas de acumulación: promociones no acumulables (is_stackable == 0) no pue \
  den combinarse con ningún otro código; promociones acumulables (is_stackable == 1) se  \
  componen multiplicativamente. Elige la opción válida que otorgue el mayor descuento al \
   cliente antes del límite de margen.
   - Limita net_revenue_usd para que gross_margin_pct nunca caiga por debajo de max(cont \
  ract_min_margin_floor_pct, pisos de promociones aplicadas), marcando margin_floor_clam \
  ped = True.
2. En inventory_atp.py:
   - Calcula raw_atp_qty = on_hand_qty - reserved_qty - safety_stock_qty - unprocessed_w \
  ebhook_delta_qty.
   - Define true_atp_qty = max(0, raw_atp_qty) y oversell_risk = (true_atp_qty < require \
  d_qty) o (raw_atp_qty <= 0).
```

3. Prompt en Línea `Cmd+I` (Edición Rápida en `inventory_atp.py`)

Selecciona la función **`calculate_location_atp`** en **`src/services/inventory_atp.py`**, presiona **`Cmd+I`** (macOS) / **`Ctrl+I`** (Windows/Linux) y escribe:

```
Valida gtin_14 usando validate_gs1_gtin14(), calcula raw_atp = on_hand - reserved - safe \
  ty_stock - webhook_delta, limita true_atp = max(0, raw_atp) y establece oversell_risk  \
  = (true_atp < required_qty) o (raw_atp <= 0).
```

4. Prompt de Implementación y Verificación

```
Implementa las correcciones completas en @src/services/promo_pricing.py y @src/services/ \
  inventory_atp.py siguiendo el plan, y luego ejecuta ./labs/lab-01-margin-inventory-tri \
  age/verify.sh para confirmar que las 6 pruebas en test_lab01_triage.py pasen al 100%.
```

5. Prompts Avanzados (`/grill-me`, `/browser`, `/learn`)

5A. Entrevista de Casos Borde (`/grill-me`)

```
/grill-me Entrevístame sobre casos borde de acumulación promocional en retail omnicanal  \
  (por ejemplo, paquetes BOGO, descuentos de empleados combinados con liquidación y redo \
  ndeo al centavo exacto en órdenes multi-unidad).
```

5B. Verificación Visual en el Dashboard (`/browser`)

```
/browser Abre http://localhost:8002, verifica que las tarjetas KPI "1. Promo Margin Floo \
  r Guard" y "2. True ATP & VIP Oversell Radar" muestren PROTECTED y TRUE ATP ACTIVE, y  \
  prueba SKU-SONY-OLED65 con los códigos VIP25,FLASH20,EMP30 en el simulador interactivo \
  .
```

5C. Persistir Aprendizaje (`/learn`)

```
/learn Persiste la regla de que todo cálculo de precios en Retail Pulse debe usar Decima \
  l(ROUND_HALF_UP) y respetar contract_min_margin_floor_pct, y que el cálculo de ATP sie \
  mpre debe restar reserved_qty, safety_stock_qty y unprocessed_webhook_delta_qty.
```